ORACLE

Production-Ready Java Containers

Smaller, Faster, Smarter

Albert Attard

Principal Java Success Consultant

Value Engineering

October 2026



Safe harbor statement

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described for Oracle's products may change and remains at the sole discretion of Oracle Corporation.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

One speaker on stage

Albert Attard
Principal Java Success Consultant,
Oracle

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Agenda title

- Production ready
- Build and tailor the image
- Startup and memory
- Configure and operate
- Next

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Production ready

Subhead goes here



What does “production ready” mean?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Command Line Arguments

Subhead goes here



Command Line Arguments

- Command line arguments can be passed to both the JVM and to the application

```
java  [JVM launcher options]  -jar application.jar  [application args]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Command Line Arguments

Environment Variables

- `_JAVA_OPTIONS`: Global forced options
- Command line arguments
- `JDK_JAVA_OPTIONS`: Defaults for `java`
- `JAVA_TOOL_OPTIONS`: JVM-wide tool injection
- `JAVA_OPTS`: Script-only convention

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Command Line Arguments

- `JDK_JAVA_OPTIONS`: Prepends default options to the `java` launcher command line

```
JDK_JAVA_OPTIONS="-Ddemo.env=workshop -XX:MaxRAMPercentage=75"
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



Command Line Arguments

- `JAVA_TOOL_OPTIONS`: Injects JVM options when the VM is created, mainly for tools, agents, or indirect launches.

```
JAVA_TOOL_OPTIONS="-javaagent:/opt/agent.jar -Dmonitoring.enabled=true"
```

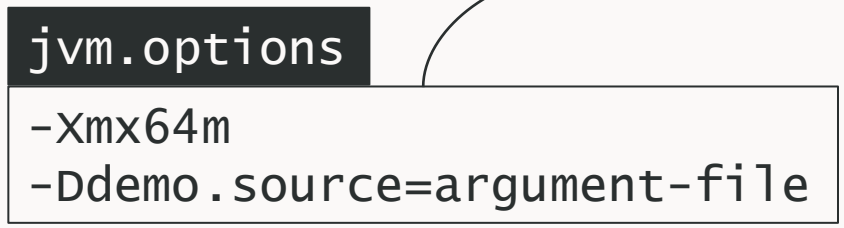

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Command Line Arguments

Argument files: @filename

- `@filename` expands JVM launcher arguments from a file at that exact command-line position

```
java  @jvm.options  -jar application.jar  [application args]
```



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

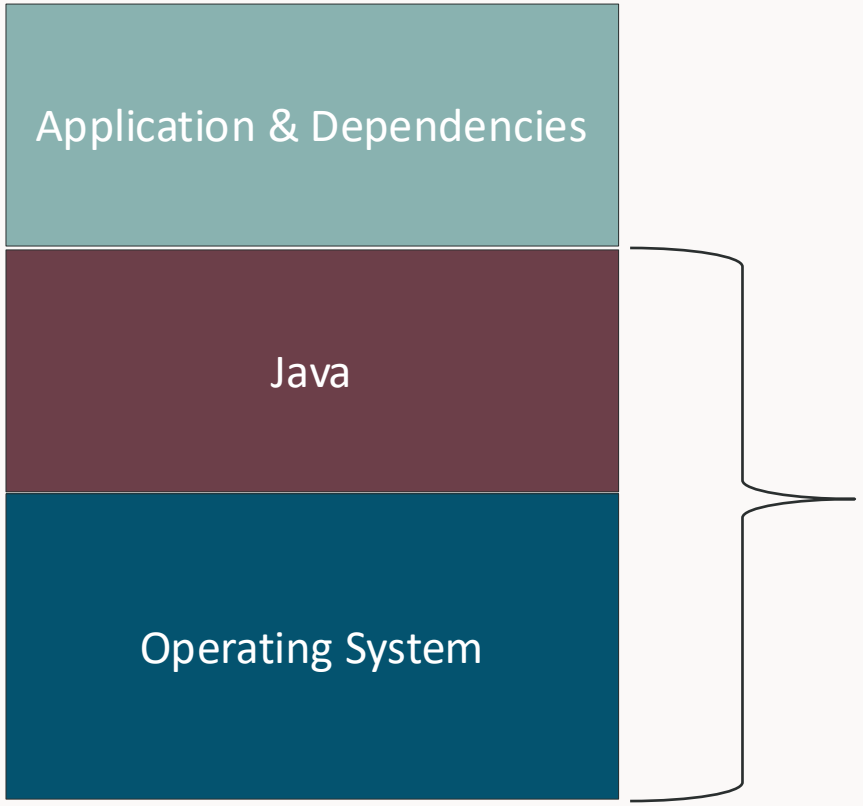
Your container foundation



What’s my container built on?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Your container foundation



The container’s foundation

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

ORACLE Oracle Container Registry

Sign In

Java Repositories

Short URL : https://container-registry.oracle.com/ords/ocr/ba/java

Repository	Description
jdk	Oracle Java Development Kit (JDK)
serverjre	Oracle Java SE (Server JRE)

1 - 2

Repository	Description	Open Source Terms and Restrictions
openjdk	Oracle OpenJDK	The container image you have selected and all of the software that it contains is licensed under one or more open source license that are provided in the container image. Your use of the container is subject to the terms of those licenses.

1 - 1

Repository	Description	Oracle JDK Container Images License
jdk-no-fee-term	Java SE Development Kit (JDK) released under Oracle's No-Fee Terms and Conditions	Your use of the Oracle Java Development Kit (“Oracle JDK”) container images, including any software packages, data or other content provided as part of Oracle JDK container images, is governed by (1) the open source or other license terms included as part of the image; and (2) the separate terms applicable to the software packages included with the Oracle JDK container images as listed in Table 1 below ((1) and (2), collectively, the “Terms of Use”). The following software packages, if included, are licensed to You under the separate terms specified for the applicable software packages as follows: TABLE 1: Package Name License Oracle JDK (Java SE) https://java.com/freeuselicense Oracle Linux https://www.oracle.com/downloads/licenses/oracle-linux-license.html By launching or using the Oracle JDK container images You agree to be bound by the Terms of Use. If You are agreeing to the Terms of Use on behalf of a company or other legal entity, You represent that You have the authority to bind it to the Terms of Use and, in such event, "You" and "Your" as used in the Terms of Use shall refer to such entity. If You are not willing to be bound by the Terms of Use do not launch or access the

https://container-registry.oracle.com/ords/ocr/ba/java

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Your container foundation

- Image tags are mutable.
- The same tag can point to a different image over time.
- Rebuilding with the same image tag can use different base-image content and introduce unexpected changes.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Your container foundation

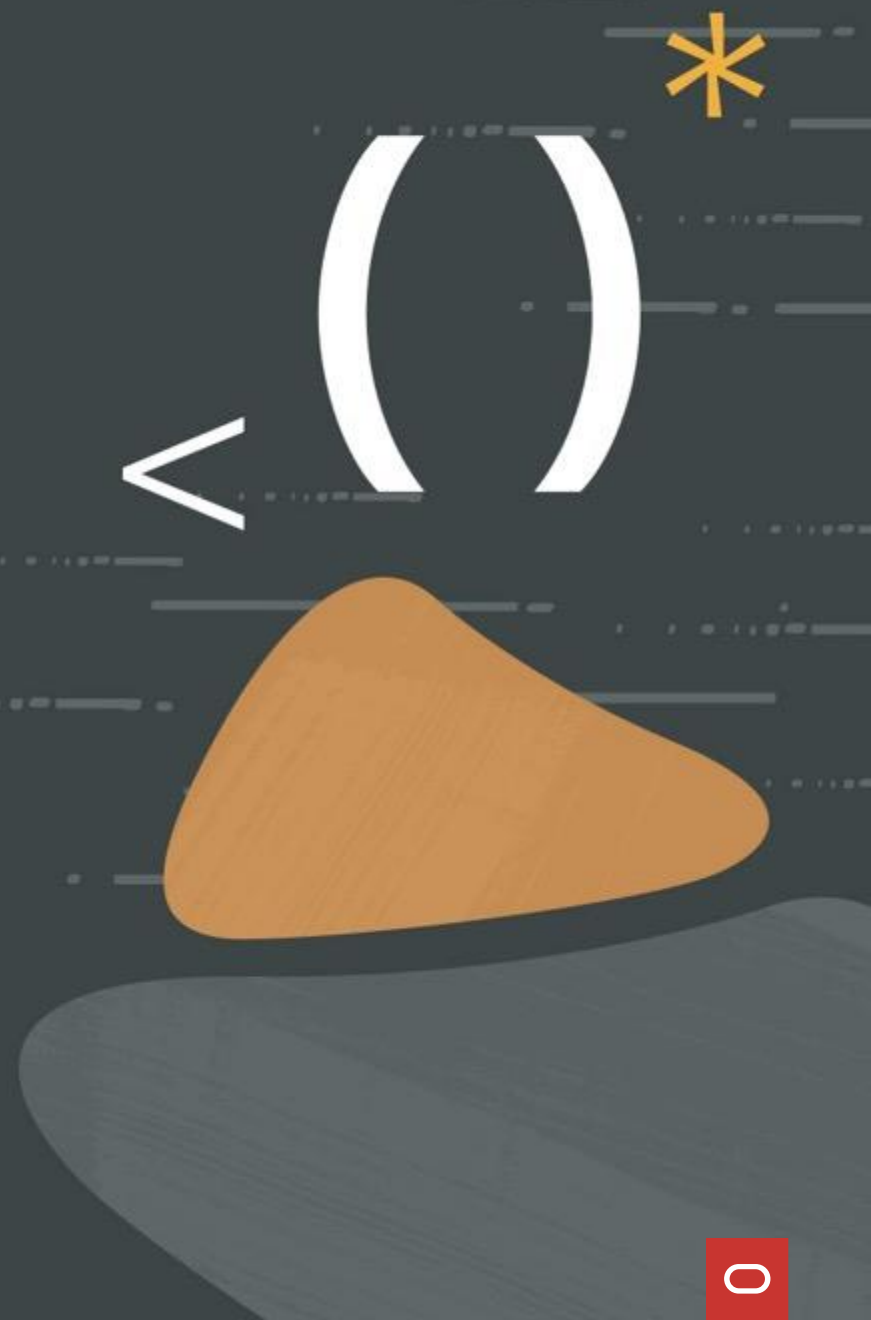
```
crane digest 'container-registry.oracle.com/java/jdk:25.0.4.1'
sha256:947650420cbf1a8d0ade992ed0cdc2edab4f8d4da1b80f501a56d3392e09e921
```

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1@sha256:947…921
```

A digest pins image content, but does not guarantee its continued availability.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Build an efficient image



What happens when code changes?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Build an efficient image

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

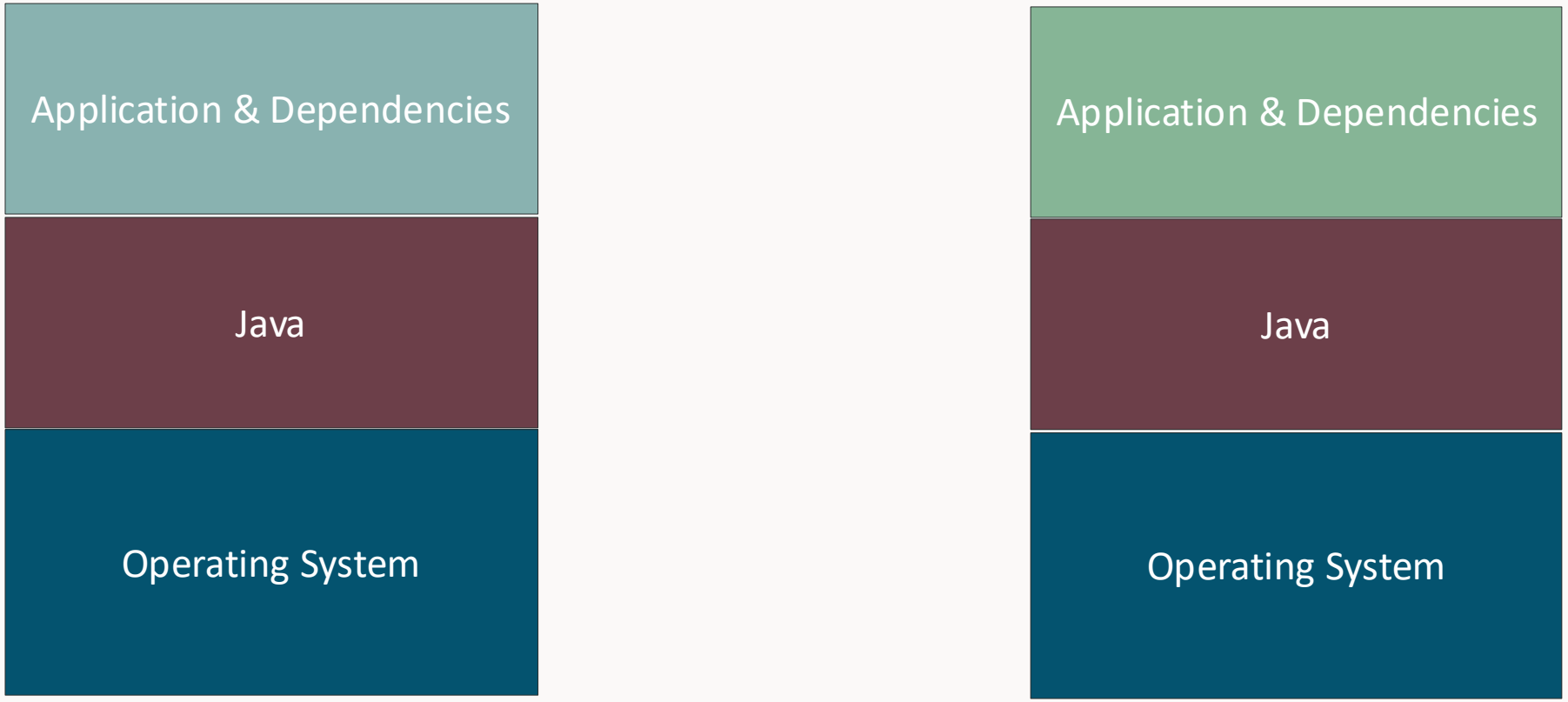
ARG JAR_FILE=./target/application-*.jar
COPY ${JAR_FILE} app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

This is a fat JAR file that includes the application and its dependencies

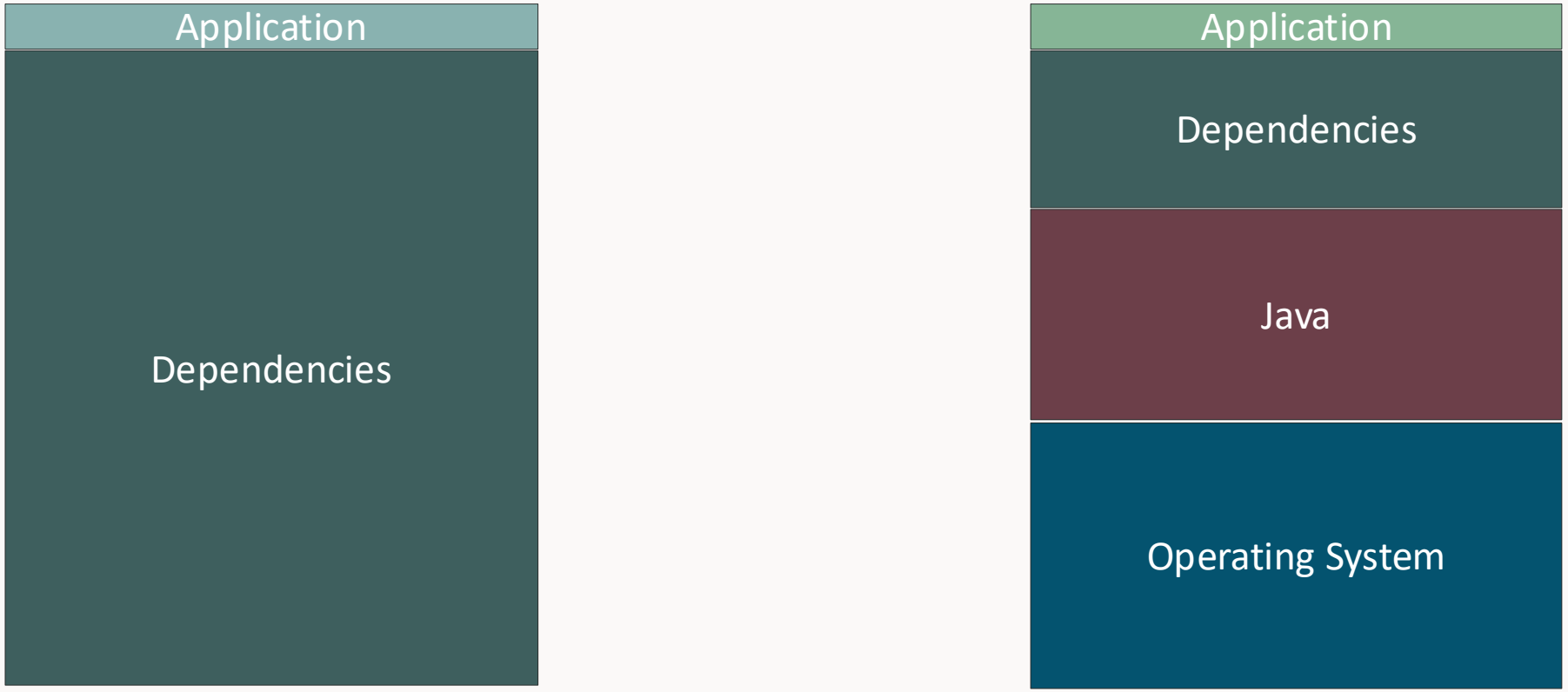
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Build an efficient image



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Build an efficient image



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Build an efficient image



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Build an efficient image

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1 AS builder
WORKDIR /opt/app

COPY ./target/application-*.jar app.jar
RUN java -Djarmode=tools -jar 'app.jar' \
  extract --layers --destination 'extracted'
```

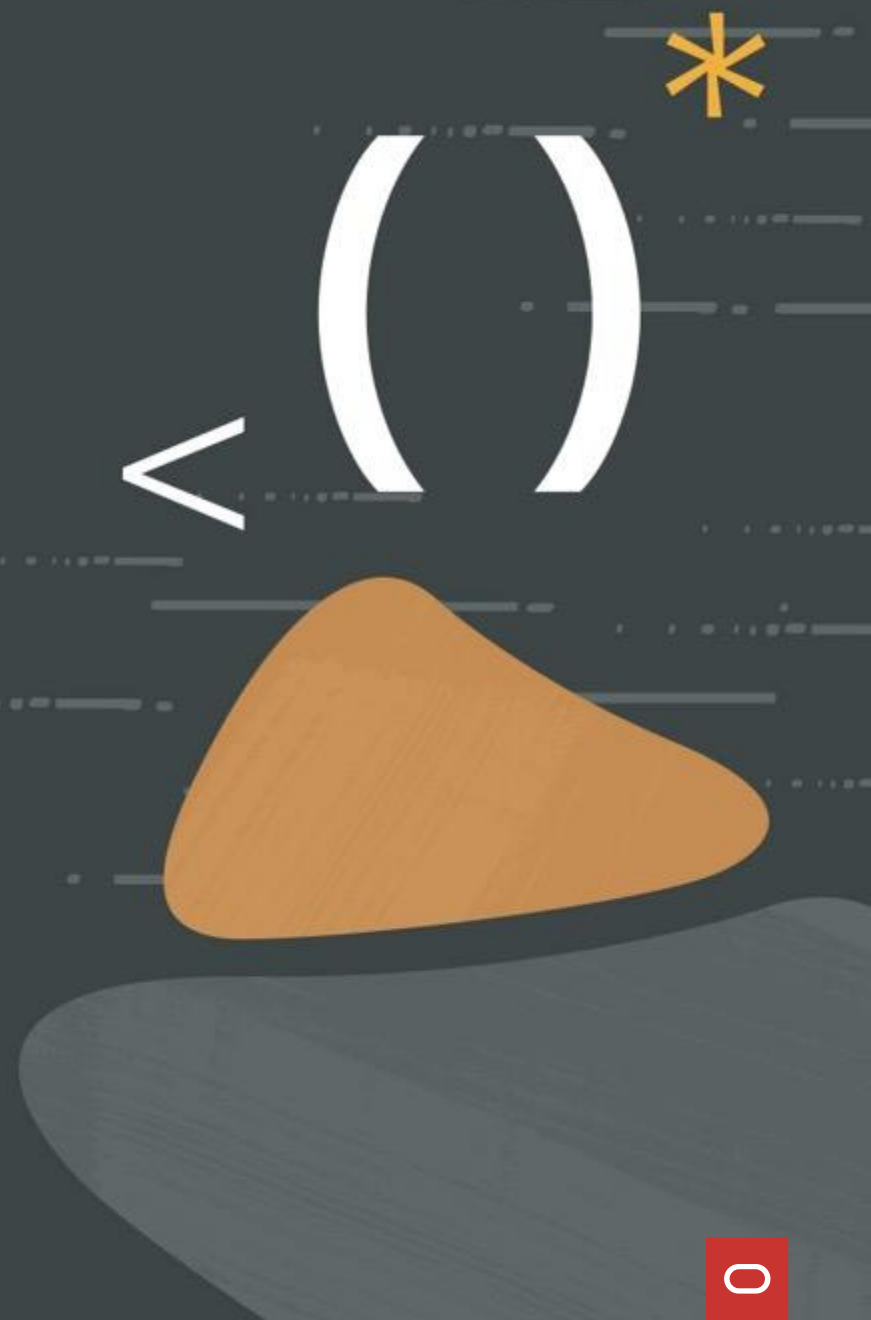
```
FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

COPY --from=builder /opt/app/extracted/dependencies/           ./
COPY --from=builder /opt/app/extracted/spring-boot-loader/     ./
COPY --from=builder /opt/app/extracted/snapshot-dependencies/  ./
COPY --from=builder /opt/app/extracted/application/            ./

ENTRYPOINT ["java", "-jar", "app.jar"]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Tailor the runtime with jlink



Is your runtime carrying dead weight?

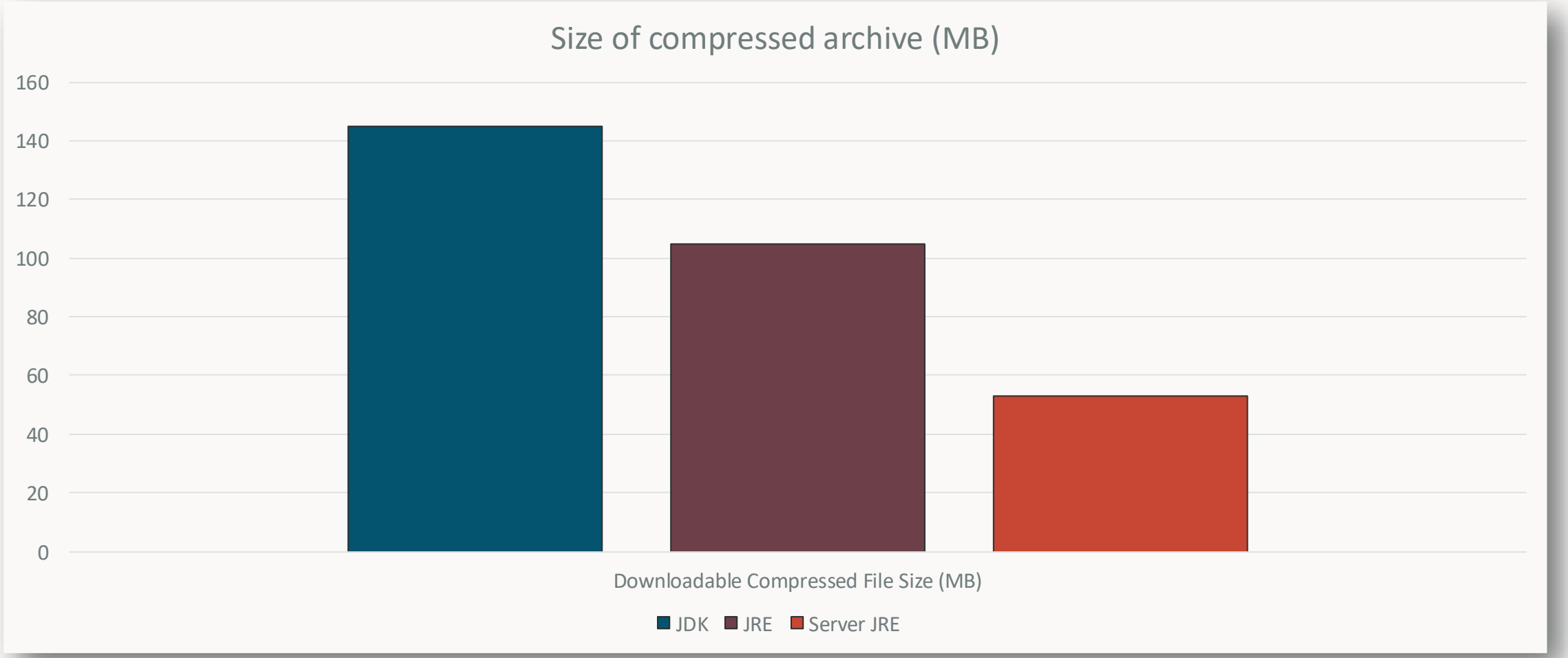
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java 8 distributions

- Java SE Development Kit (JDK)
- Java SE Runtime Environment (JRE)
- Server Java SE Runtime Environment (Server JRE)

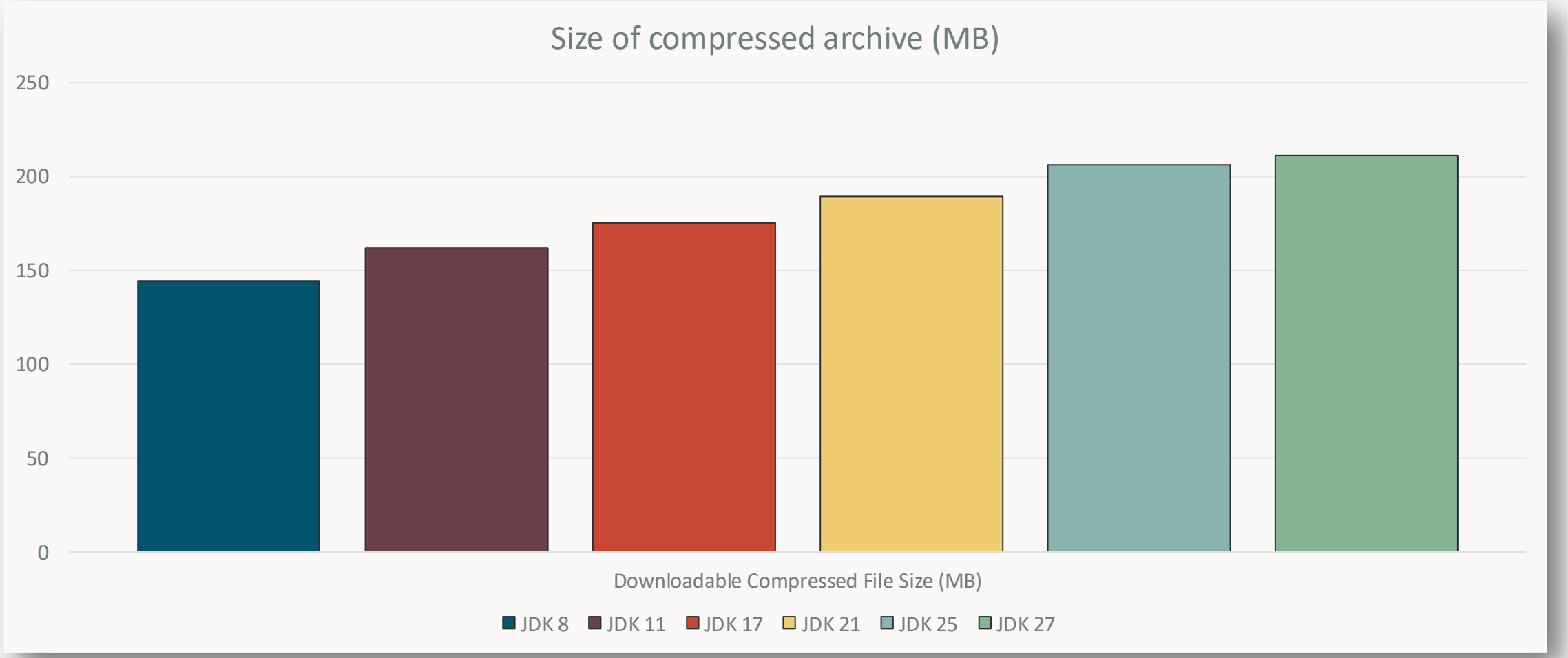
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java 8 distributions



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

JDK size



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java Distributions

- Java 8 and earlier: the JDK included a `jre/` directory, and a separate JRE distribution was available.
- Java 9: the JDK and JRE became modular runtime images; `jlink` was introduced.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java Distributions

- Java 9 onwards: Oracle no longer provides separate JRE downloads.
- Use `jlink` to create an application-specific runtime image containing only the required modules and dependencies.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

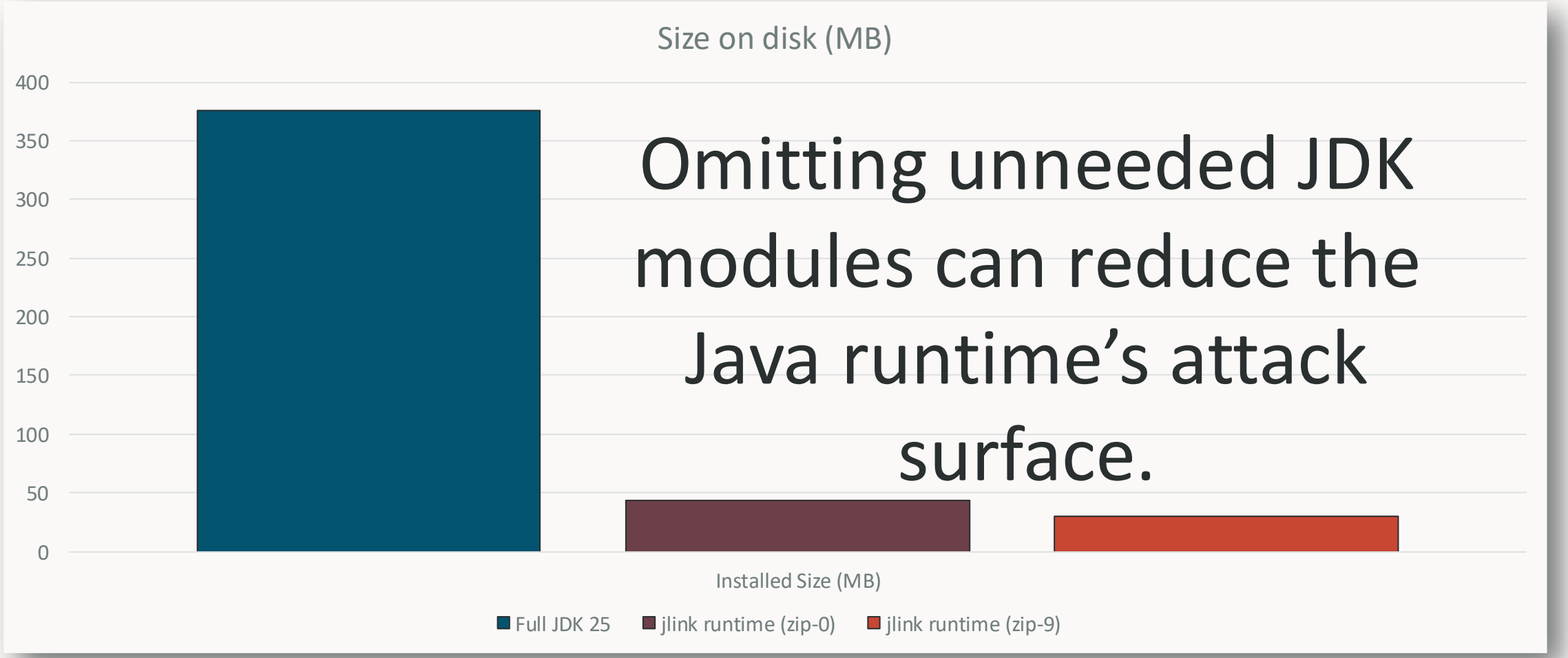
Minimal jlink runtime

```
jlink \
  --output './custom-image' \
  --no-header-files \
  --no-man-pages \
  --add-modules java.base,... \
  --strip-debug \
  --compress=zip-9
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Minimal jlink runtime

Runtime image size: `java.base` only



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Tailor the runtime with jlink

```
ARG JAVA_MODULES=java.base

RUN dnf install -y binutils && dnf clean all
RUN jlink \
      --output '/opt/jre' \
      --no-header-files \
      --no-man-pages \
      --strip-debug \
      --add-modules "${JAVA_MODULES}" \
      --compress=zip-9

FROM container-registry.oracle.com/os/oraclelinux:10-slim
WORKDIR /opt/app

# Copy the custom Java image
COPY --from=builder /opt/jre /opt/jre
```

~ 41MB

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Observability

```
ARG JAVA_MODULES=java.base,…,jdk.jcmd,jdk.jfr

RUN dnf install -y binutils && dnf clean all
RUN jlink \
      --output '/opt/jre' \
      --no-header-files \
      --no-man-pages \
      --strip-debug \
      --add-modules "${JAVA_MODULES}" \
      --compress=zip-9
```

```
FROM container-registry.oracle.com/os/oraclelinux:10-slim
WORKDIR /opt/app

# Copy the custom Java image
COPY --from=builder /opt/jre /opt/jre
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

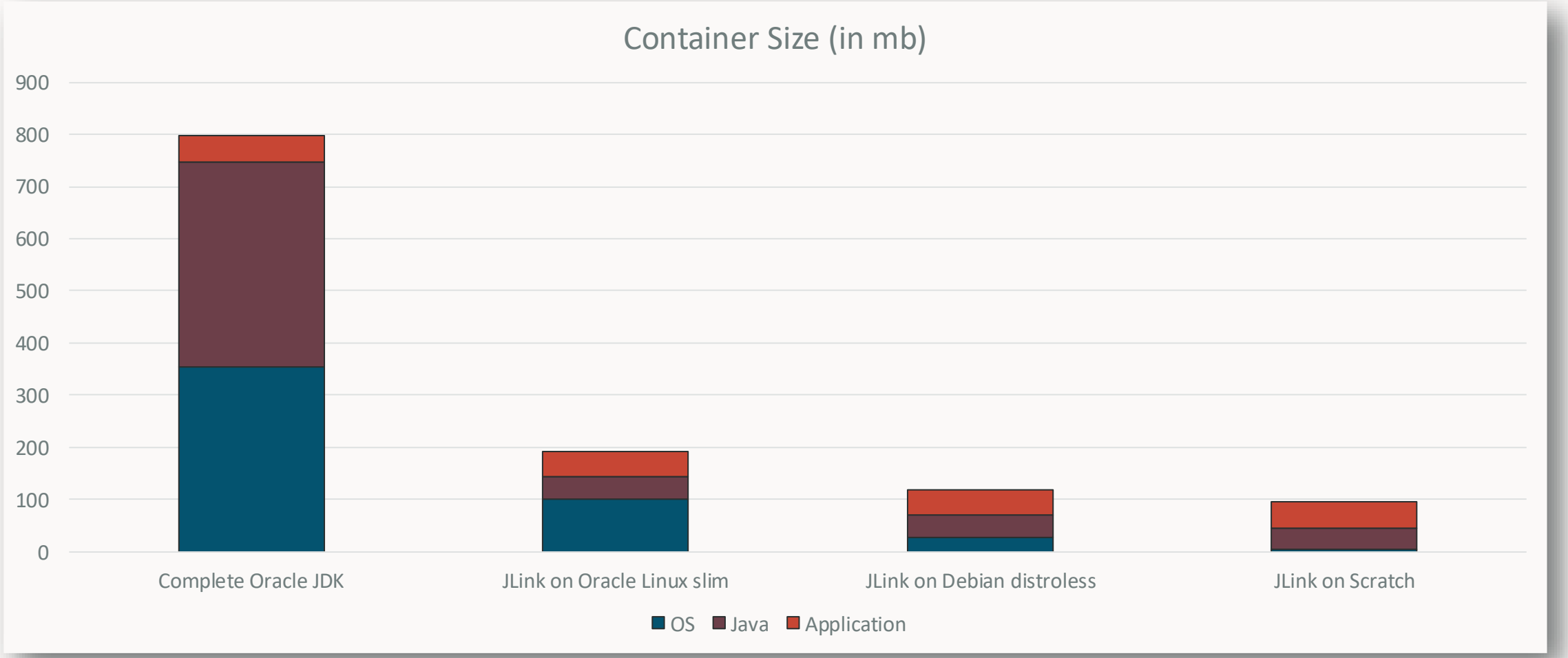
Optimise the OS image



What does your OS image expose?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Optimise the OS image



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Optimise the OS image

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1 AS builder

ARG JAVA_MODULES=java.base

# binutils are needed by the `--strip-debug` command line option
RUN dnf install -y binutils && dnf clean all
RUN jlink \
      --output '/opt/jre' \
      --no-header-files \
      --no-man-pages \
      --strip-debug \
      --add-modules "${JAVA_MODULES}" \
      --compress=zip-9

FROM container-registry.oracle.com/os/oraclelinux:10-slim ~101MB
WORKDIR /opt/app
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Optimise the OS image

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1 AS builder

ARG JAVA_MODULES=java.base

# binutils are needed by the `--strip-debug` command line option
RUN dnf install -y binutils && dnf clean all
RUN jlink \
      --output '/opt/jre' \
      --no-header-files \
      --no-man-pages \
      --strip-debug \
      --add-modules "${JAVA_MODULES}" \
      --compress=zip-9

FROM gcr.io/distroless/java-base-debian13:nonroot
WORKDIR /opt/app
```

~ 28MB

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Optimise the OS image

```
# Copy only native dependencies outside the custom runtime from the same
# Oracle Linux image as the JDK. Preserve their absolute paths for the loade
ENV LD_LIBRARY_PATH=/opt/jre/lib:/opt/jre/lib/server:/opt/jre/lib/jli
RUN mkdir -p /opt/rootfs && \
  /* Find and copy the native libraries needed by the custom JRE for the
     scratch image. */

FROM scratch
WORKDIR /opt/app

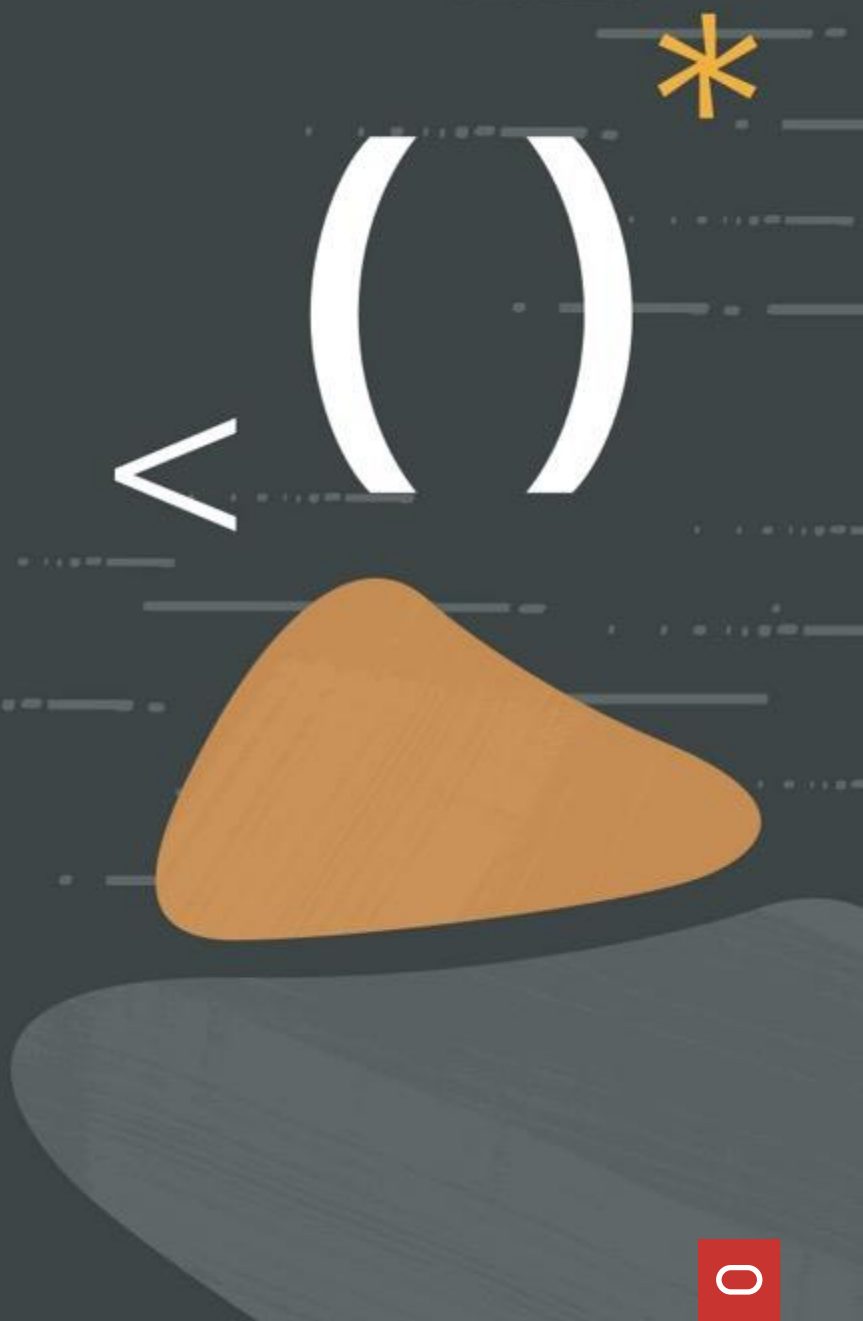
COPY --from=builder /opt/rootfs/ /
COPY --from=builder /opt/jre /opt/jre

ARG JAR_FILE=./target/applicaion-*.jar
COPY ${JAR_FILE} app.jar
```

~ 6MB

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

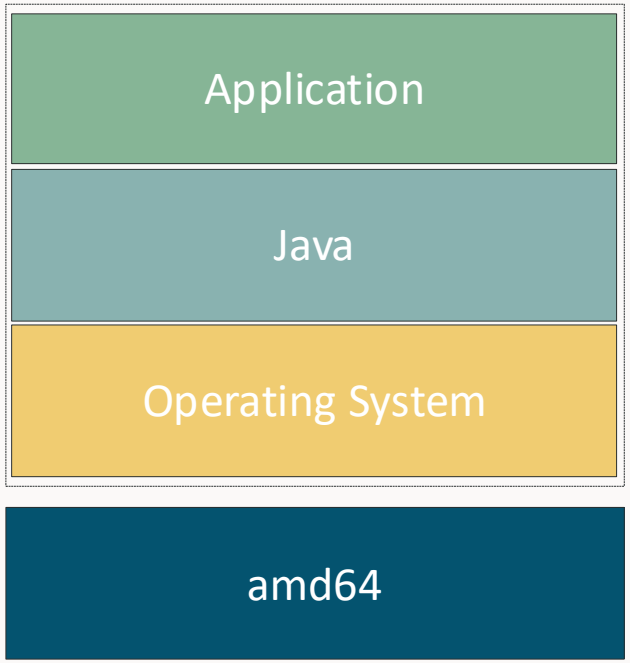
Configure one image for different environments



One image tag for ARM64 and AMD64?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Configure one image for different environments





Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Configure one image for different environments

```
➜  ~ skopeo inspect --raw 'docker://container-registry.oracle.com/os/oraclelinux:10-slim' | jq
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.index.v1+json",
  "manifests": [
    {
      "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "digest": "sha256:fd97561c070364cd3e640b18848f28a3184f0126ec4ab062351c7dab1a147be2",
      "size": 429,
      "platform": {
        "architecture": "amd64",
        "os": "linux"
      }
    },
    {
      "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "digest": "sha256:e1fc77b5efd9a7a6af6e2b0912c0d3810ee73715d3168f18064d407bc6308f5f",
      "size": 429,
      "platform": {
        "architecture": "arm64",
        "os": "linux"
      }
    }
  ]
}
➜  ~
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Configure one image for different environments

```
➜  ~ skopeo inspect --raw 'docker://container-registry.oracle.com/java/jdk:25.0.4.1' | jq
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.docker.distribution.manifest.list.v2+json",
  "manifests": [
    {
      "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "size": 1099,
      "digest": "sha256:06d115c14b85dc23030635780c6a587096dfb0583085c31d36be634b7efe49f2",
      "platform": {
        "architecture": "arm64",
        "os": "linux"
      }
    },
    {
      "mediaType": "application/vnd.docker.distribution.manifest.v2+json",
      "size": 1099,
      "digest": "sha256:c2fde8cea2f620b6b7dafc53fa9a4ac3f88ed41ad1ad35e085c56decdfc96d12",
      "platform": {
        "architecture": "amd64",
        "os": "linux"
      }
    }
  ]
}
➜  ~
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Configure one image for different environments

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

COPY ./target/application-*.jar app.jar

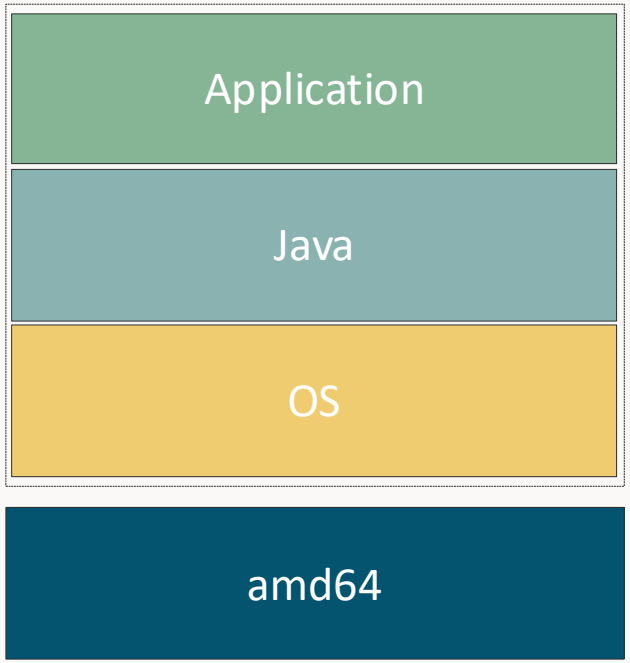
EXPOSE 8080

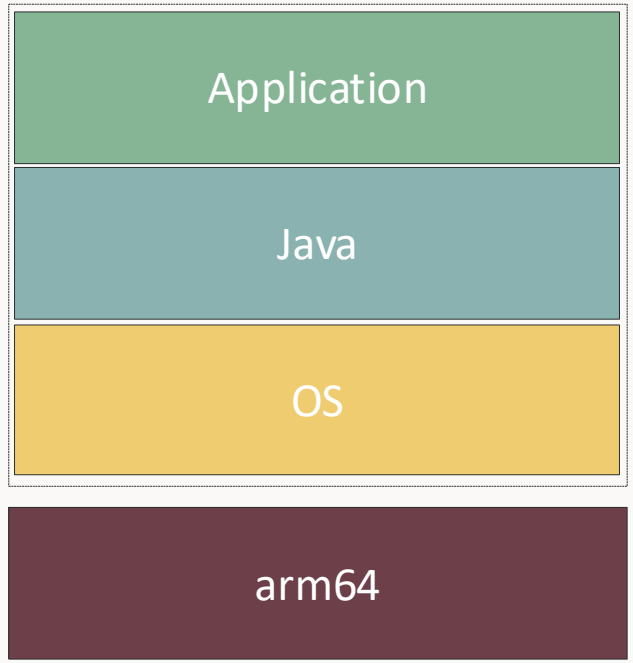
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
docker buildx build \
  --platform 'linux/amd64,linux/arm64' \
  --file './containers/Dockerfile' \
  --tag 'demo:multi-architecture-image-builds-oci' \
  --output 'type=oci,dest=./target/multi-architecture-image-builds.tar' \
  .
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

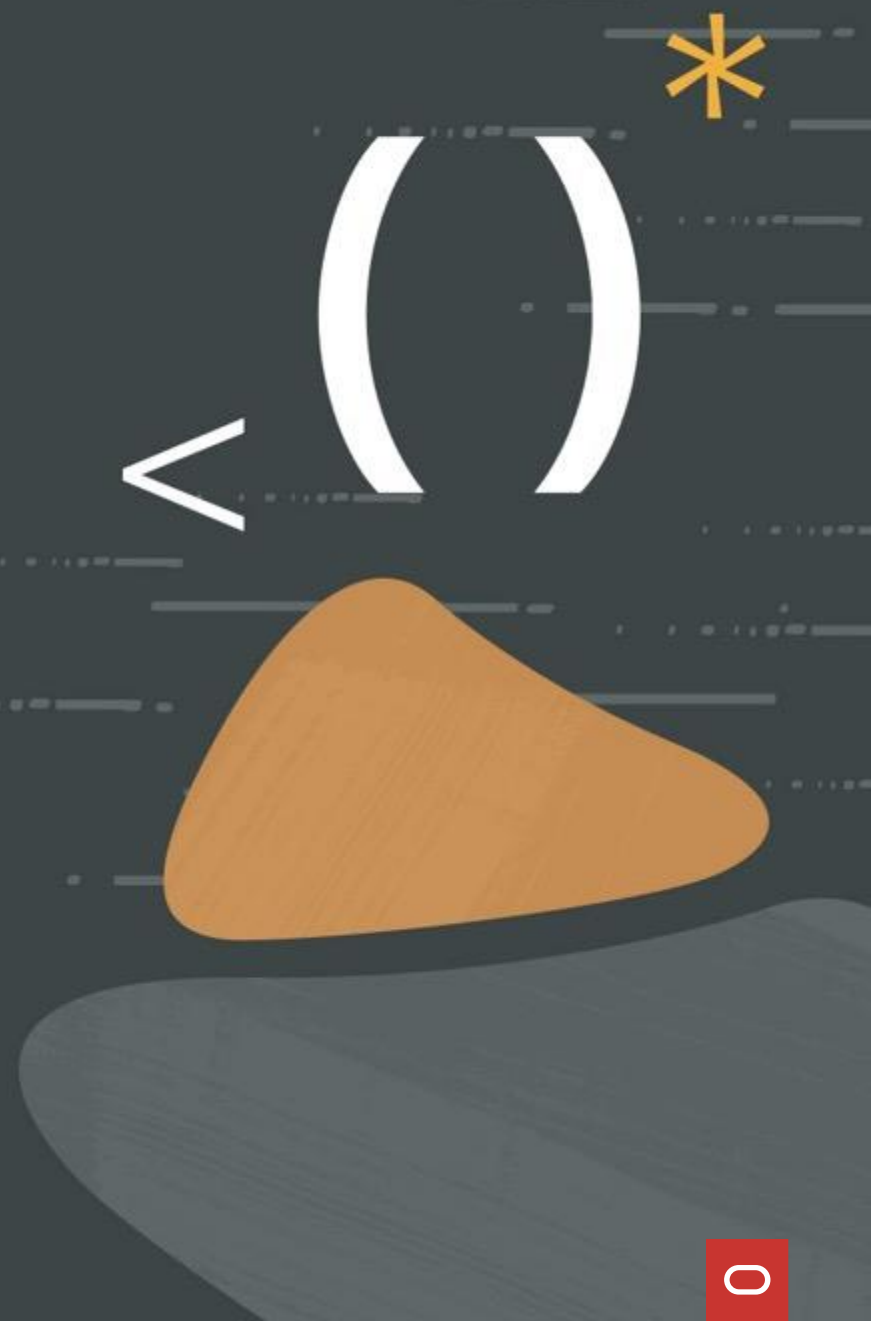
Configure one image for different environments





Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches



What happens at startup?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

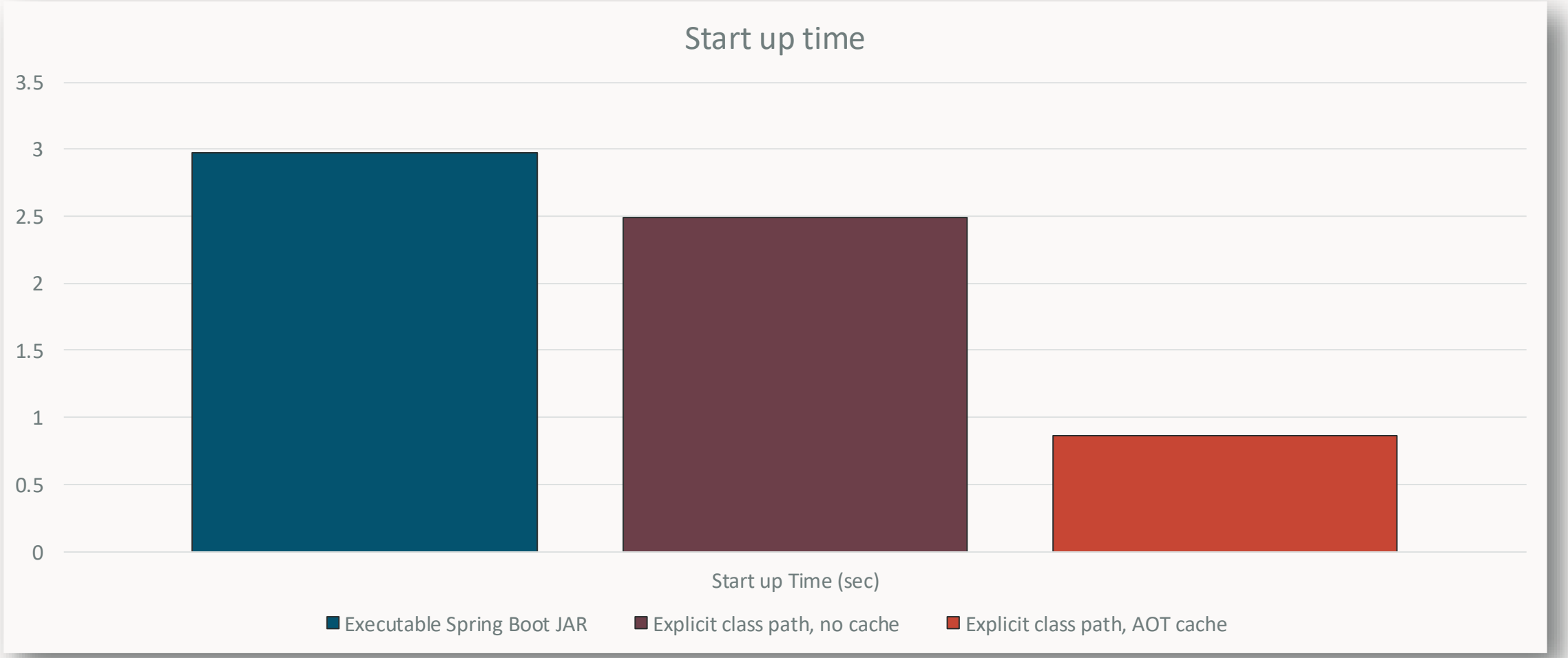
Improve startup with CDS and Java 25 AOT caches



The AOT cache speeds up class loading and linking.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

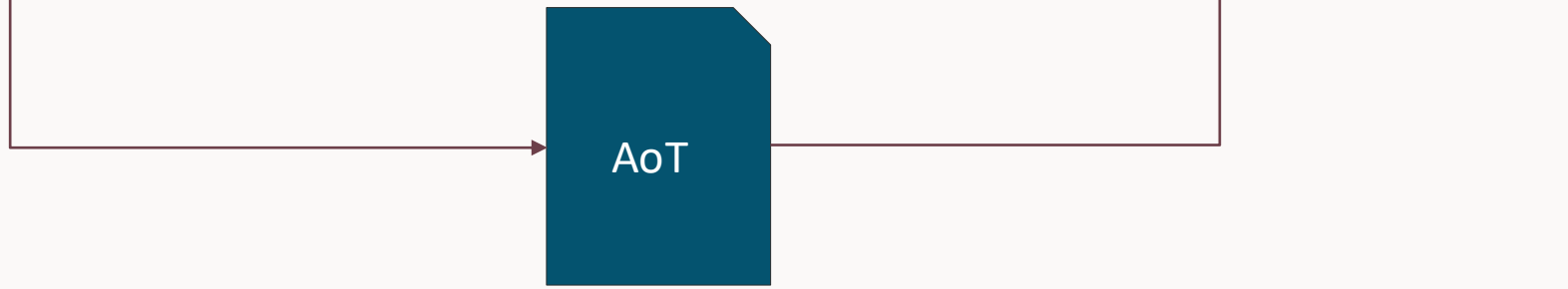
Improve startup with CDS and Java 25 AOT caches

Generate

```
java \
  -XX:AOTCacheOutput=app.aot \
  …
```

Use

```
java \
  -XX:AOTCache=app.aot \
  …
```



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches

- The AOT cache is tied to the application’s classpath.
- Changing the order can change which JAR supplies a class or resource, so the JVM cannot assume its cached loading and linking decisions are valid.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1 AS builder
WORKDIR /opt/app

COPY ./target/application-*.jar app.jar

# The AOT cache is tied to the exact class path used during training. Even a
# different dependency JAR order invalidates the cache, so write the class p
# to java.args and use that same argument file for cache creation and startu
RUN mkdir extracted \
    && (cd extracted && jar -xf ../app.jar) \
    && jar --create --file application.jar -C extracted/BOOT-INF/classes . \
    && printf '…' > java.args \
    && java \
        -Dspring.context.exit=onRefresh \
        -XX:AOTCacheOutput=app.aot \
        @java.args
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches

```
        -XX:AOTCacheOutput=app.aot \
        @java.args

FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

COPY --from=builder /opt/app/extracted/BOOT-INF/lib ./extracted/BOOT-INF/lib
COPY --from=builder /opt/app/application.jar        ./application.jar
COPY --from=builder /opt/app/java.args              ./java.args
COPY --from=builder /opt/app/app.aot                ./app.aot

ENV JDK_JAVA_OPTIONS="-XX:AOTCache=app.aot -XX:AOTMode=on"

# Reuse java.args so runtime sees the same class path order used to create t
# AOT cache.
ENTRYPOINT ["java", "@java.args"]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches

```
        -XX:AOTCacheOutput=app.aot \
        @java.args

FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

COPY --from=builder /opt/app/extracted/BOOT-INF/lib ./extracted/BOOT-INF/lib
COPY --from=builder /opt/app/application.jar        ./application.jar
COPY --from=builder /opt/app/java.args              ./java.args
COPY --from=builder /opt/app/app.aot                ./app.aot

ENV JDK_JAVA_OPTIONS="-XX:AOTCache=app.aot -XX:AOTMode=on"

# Reuse java.args so runtime sees the same class path order used to create t
# AOT cache.
ENTRYPOINT ["java", "@java.args"]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

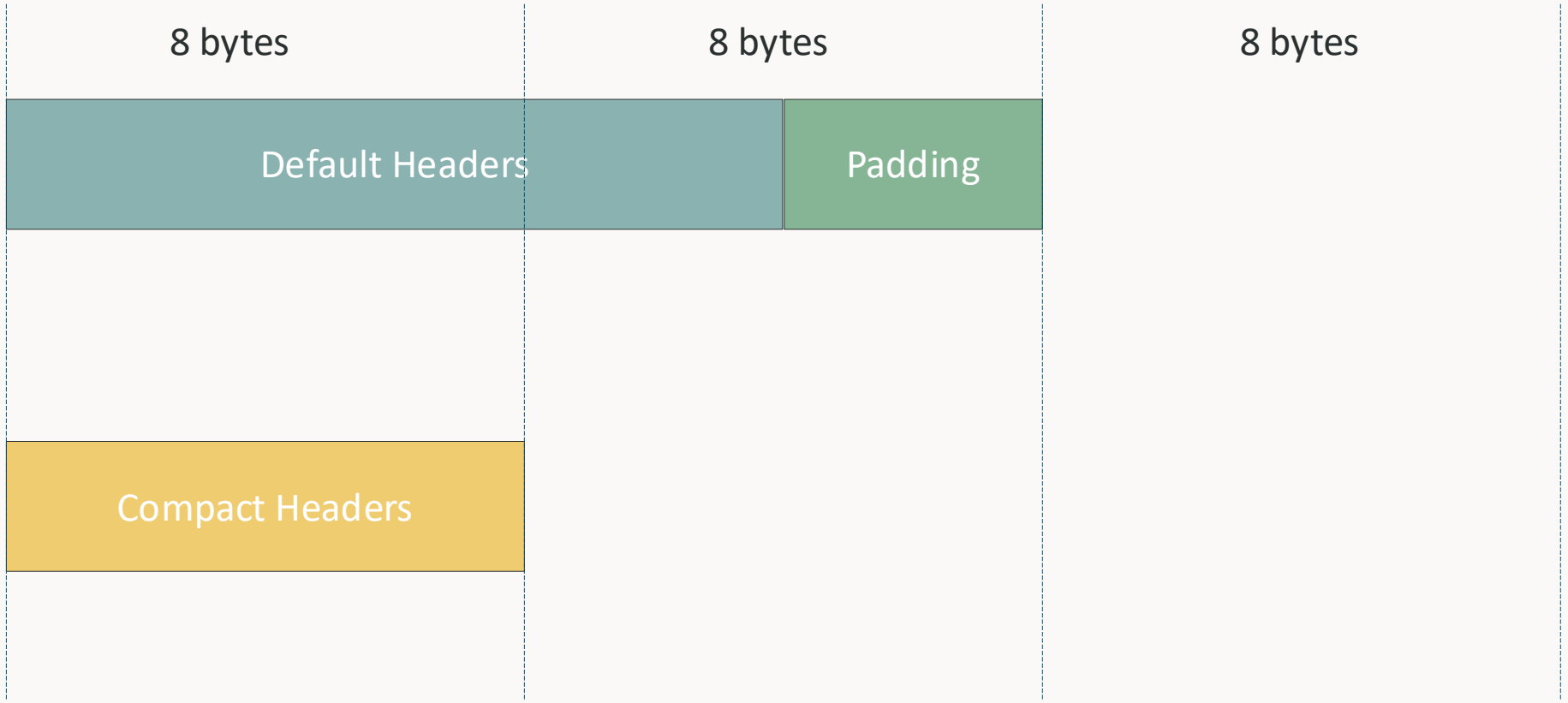
Compact object headers



How much overhead per object?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

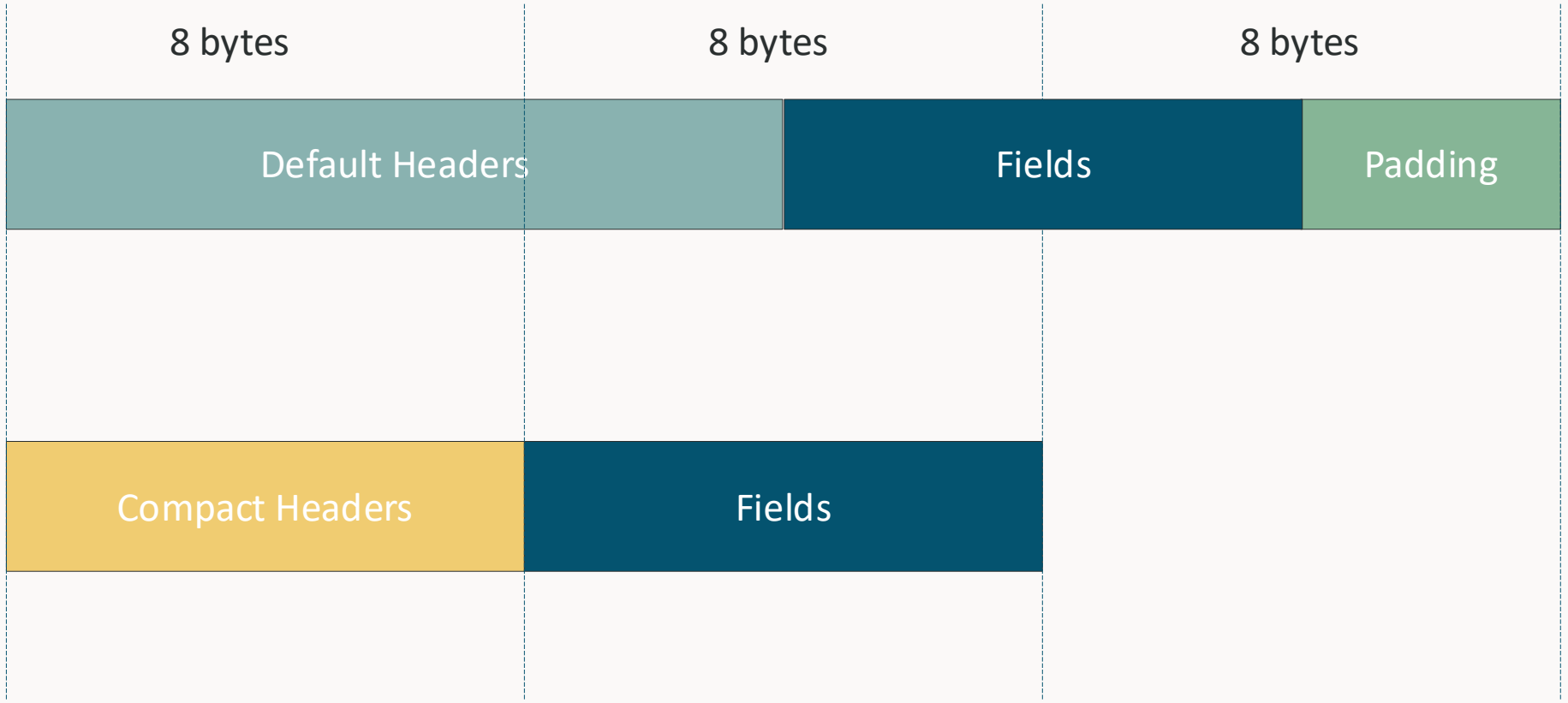
Compact object headers




```
public record Example() {}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

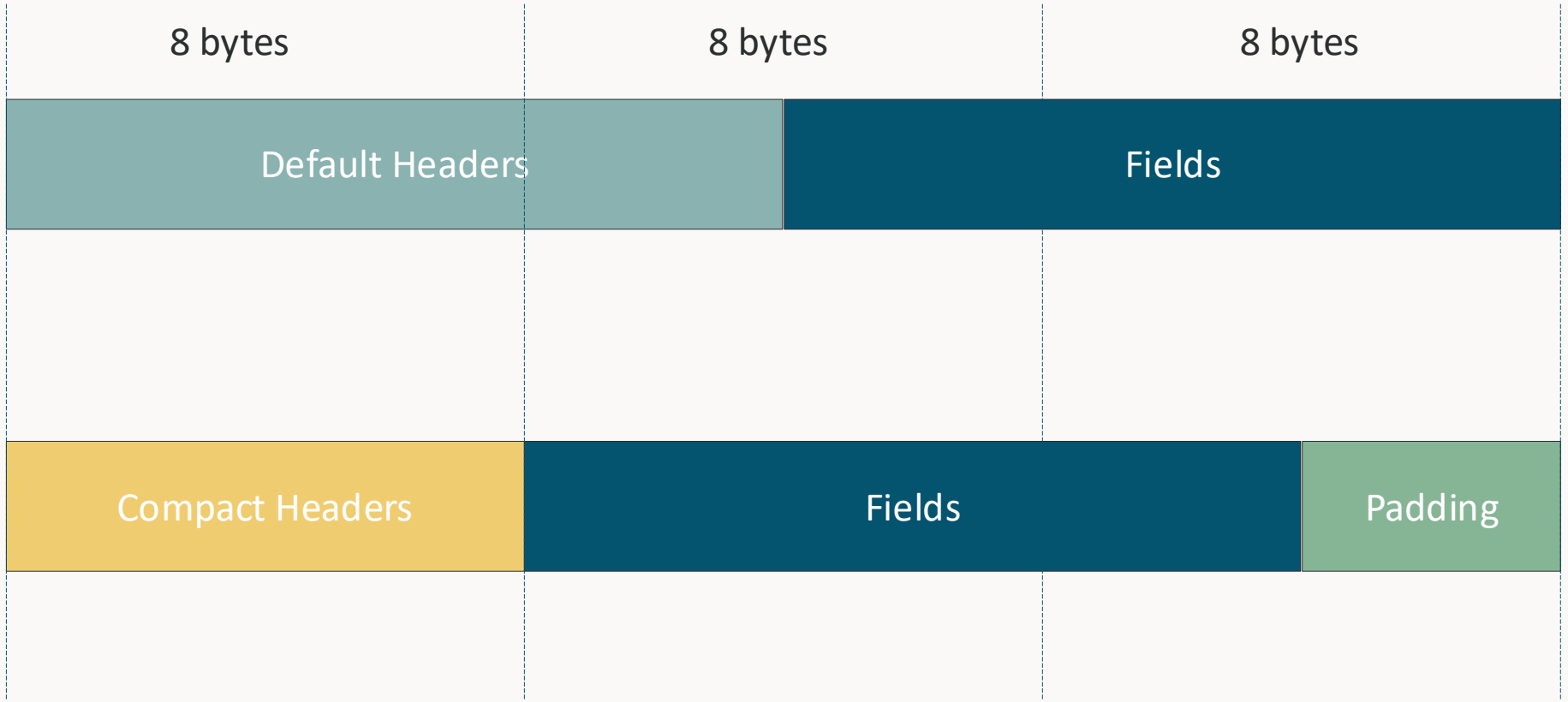
Compact object headers




```
public record Example(int a, int b) {}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

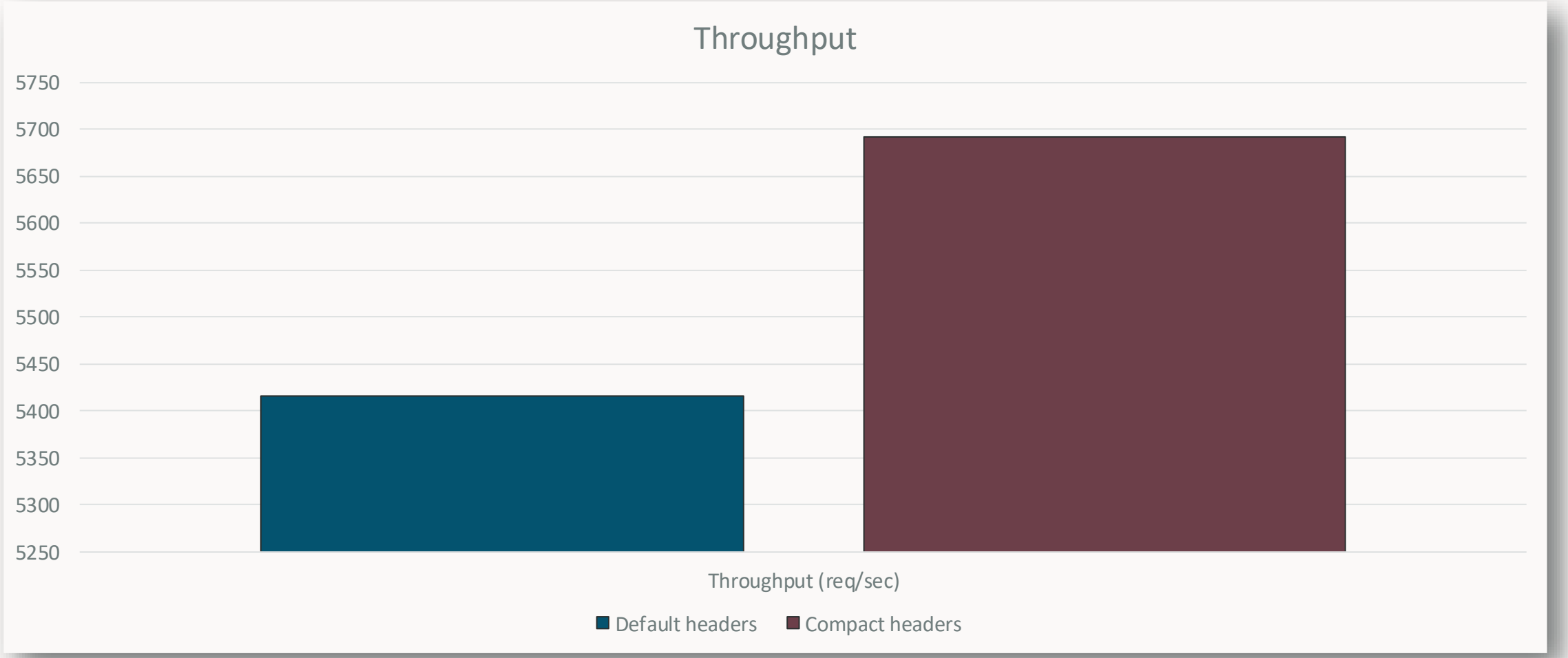
Compact object headers




```
public record Example(int a, int b, int c) {}
```

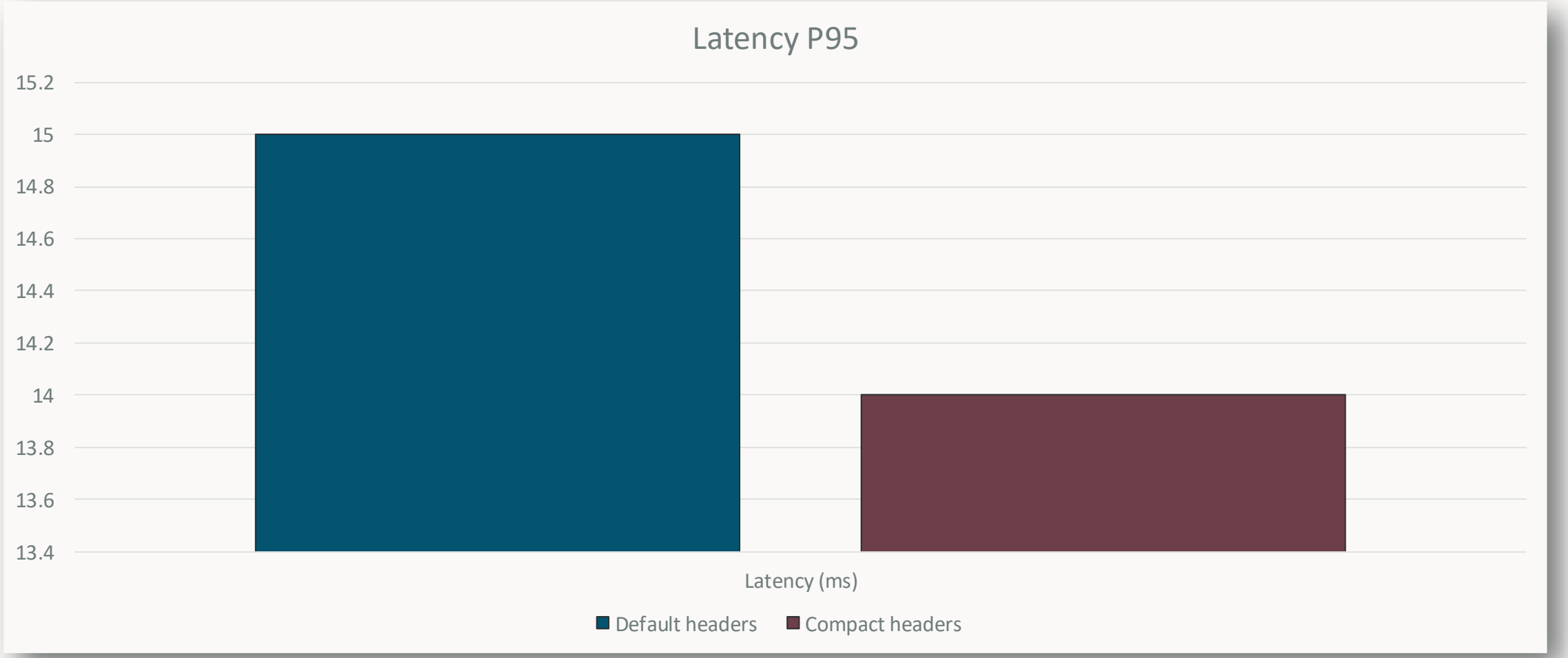

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches



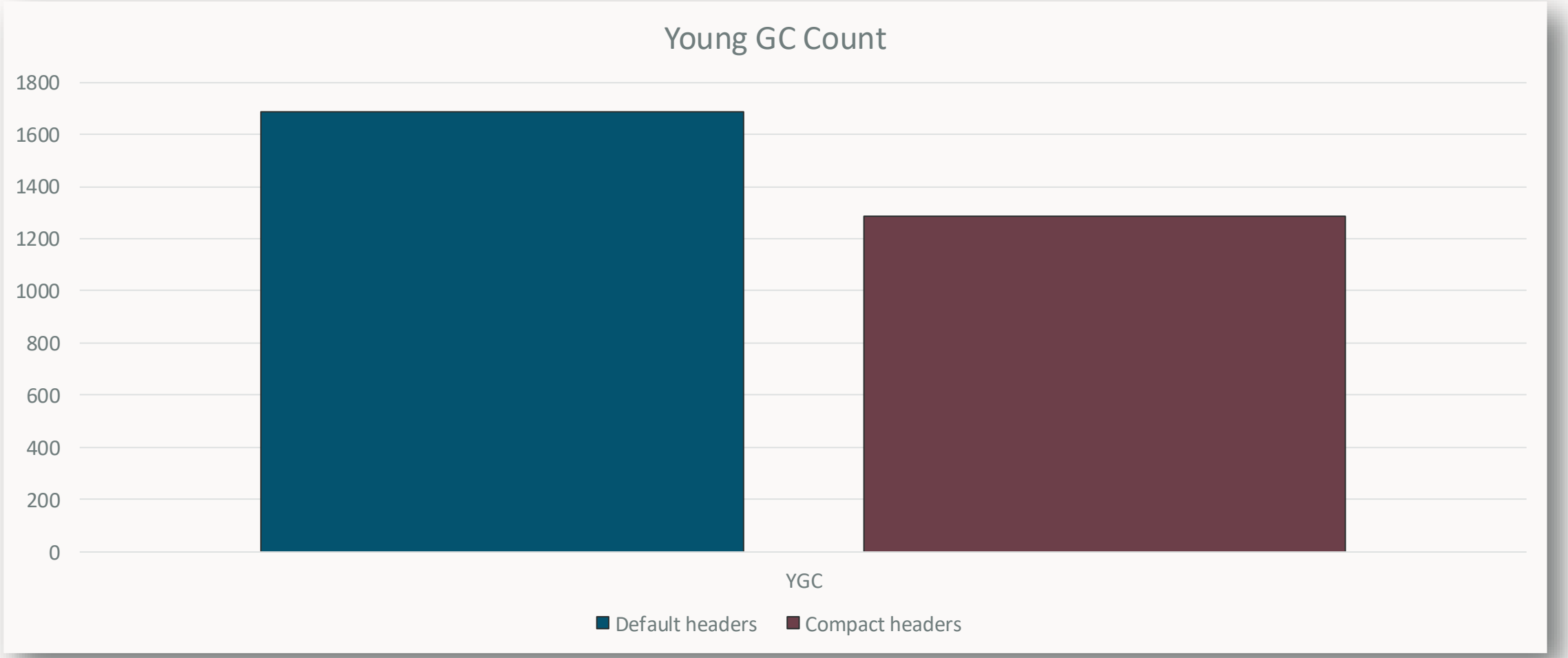
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Improve startup with CDS and Java 25 AOT caches

Java 25

```
java \
  -XX:+UseCompactObjectHeaders \
  …
```

Available but not enabled by default

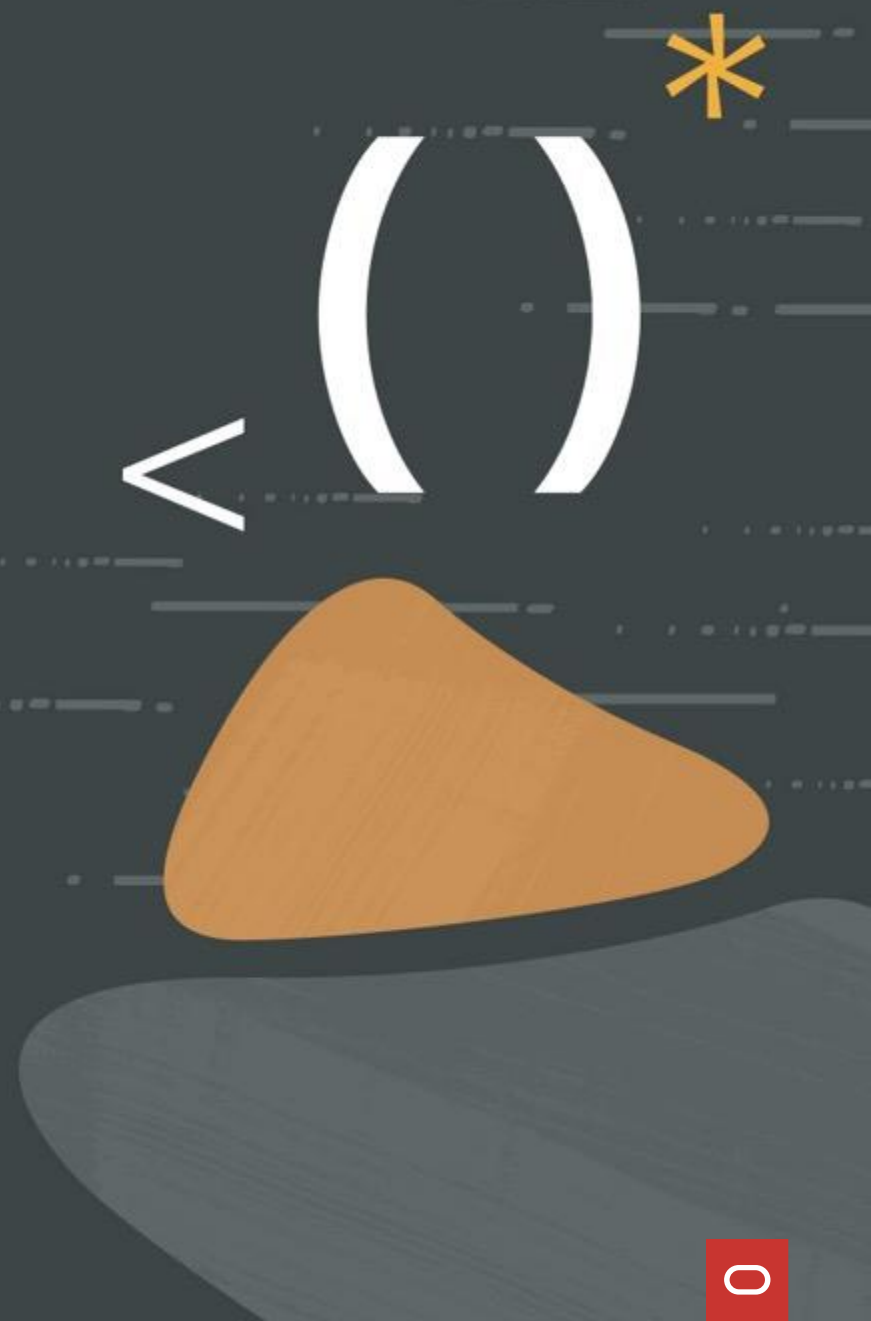
Java 27

```
java \
  -XX:-UseCompactObjectHeaders \
  …
```

Enabled by default

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java and container resource limits

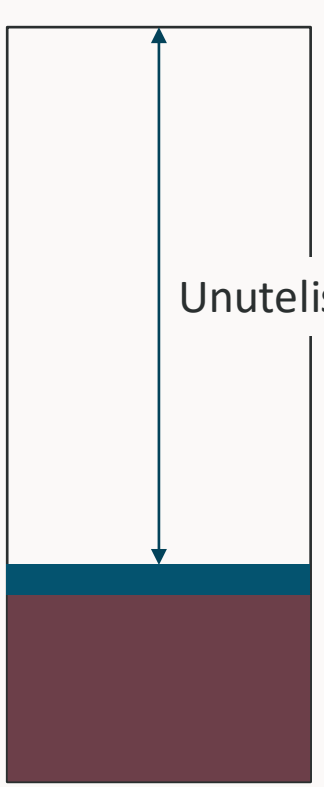


Do you know your JVM’s defaults?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java and container resource limits

```
docker run \
  --memory=1G \
  ...
```




```
java -jar app.jar
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Title and Copyright Information
Preface
1 Introduction to Garbage Collection Tuning
2 Ergonomics
3 Garbage Collector Implementation
4 Factors Affecting Garbage Collection Performance
5 Available Collectors
6 The Parallel Collector
7 Garbage-First (G1) Garbage Collector
8 Garbage-First Garbage Collector Tuning
9 The Z Garbage Collector
10 Other Considerations

application.

This section describes these default selections and behavior-based tuning. Use these defaults before using the more detailed controls described in subsequent sections.

Topics

- Garbage Collector, Heap, and Runtime Compiler Default Selections
- Behavior-Based Tuning
  - Maximum Pause-Time Goal
  - Throughput Goal
  - Footprint
- Tuning Strategy

Garbage Collector, Heap, and Runtime Compiler Default Selections

These are important garbage collector, heap size, and runtime compiler default selections:

- Garbage-First (G1) Collector on server-class machines, Serial Collector otherwise.
- The maximum number of GC threads is limited by heap size and available CPU resources
- Initial heap size of 1/64 of physical memory
- Maximum heap size of 1/4 of physical memory
- Tiered compiler, using both C1 and C2

> **Note**
>
> The VM considers machines as server-class if the VM detects two or more processors and physical memory larger than or equal to 1792 MB.

On This Page
2 Ergonomics
Garbage Collector, Heap, and Runtime Compiler Default Selections
Behavior-Based Tuning
Maximum Pause-Time Goal
Throughput Goal
Footprint
Tuning Strategy

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java and container resource limits

```
docker run \
  --rm \
  --memory=1G \
  container-registry.oracle.com/java/jdk:25.0.4.1 \
  java -XX:+PrintFlagsFinal --version \
  | grep -E 'UseSerial|MaxHeapSize|MaxRAMPercentage'

   size_t MaxHeapSize          = 268435456        {product} {ergonomic}
   double MaxRAMPercentage     = 25.000000        {product} {default}
   size_t SoftMaxHeapSize      = 268435456     {manageable} {ergonomic}
     bool UseSerialGC          = true             {product} {ergonomic}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Java and container resource limits

```
docker run \
  --rm \
  --memory=1G \
  container-registry.oracle.com/java/jdk:25.0.4.1 \
  java -XX:+PrintFlagsFinal -XX:MaxRAMPercentage=75.0 --version \
  | grep -E 'UseSerial|MaxHeapSize|MaxRAMPercentage'

   size_t MaxHeapSize          = 805306368        {product} {ergonomic}
   double MaxRAMPercentage     = 75.000000        {product} {command line}
   size_t SoftMaxHeapSize      = 805306368     {manageable} {ergonomic}
     bool UseSerialGC          = true             {product} {ergonomic}
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes



Java `OutOfMemoryError` or OOM kill?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

- The JVM throws a Java `OutOfMemoryError` because it cannot satisfy an allocation.
- The container reaches its memory limit and the OOM killer terminates the Java process.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

```
docker run \
  --name 'memory-limits' \
  --memory=256M \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-Xms256m -Xmx256m' \
  'demo:application'
```

The application will start and it will keep working until Java tries to use more memory than allocated to the container

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

```
docker run \
  --name 'memory-limits' \
  --memory=256M \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-Xms256m -Xmx256m -XX:+AlwaysPreTouch' \
  'demo:application'
```

`-XX:+AlwaysPreTouch` makes the JVM touch every page of the initial Java heap during startup, rather than waiting until the application uses those pages.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

```
docker inspect --format '{{.State.OOMKilled}} {{.State.ExitCode}}' \
  'memory-limits'
true 137
```

The Java process was OOM-killed, causing the container to stop.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

```
docker run \
  --name 'memory-limits' \
  --memory=256M \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-XX:InitialRAMPercentage=75 -XX:MaxRAMPercentage=75' \
  'demo:application'
```

At startup, the JVM sets the initial and maximum Java heap to approximately 75% of the container's 256 MiB memory limit.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

```
docker run \
  --name 'memory-limits' \
  --memory=256M \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-Xms32m -Xmx32m' \
  'demo:application'
```

An `OutOfMemoryError` does not, by itself, terminate the JVM.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Distinguish memory failure modes

```
docker run \
  --name 'memory-limits' \
  --memory=256M \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-Xms32m -Xmx32m -XX:+ExitOnOutOfMemoryError' \
  'demo:application'
```

```
  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/diagnostics/heap.hprof
```

The JVM exits when an `OutOfMemoryError` occurs, causing the container to stop.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

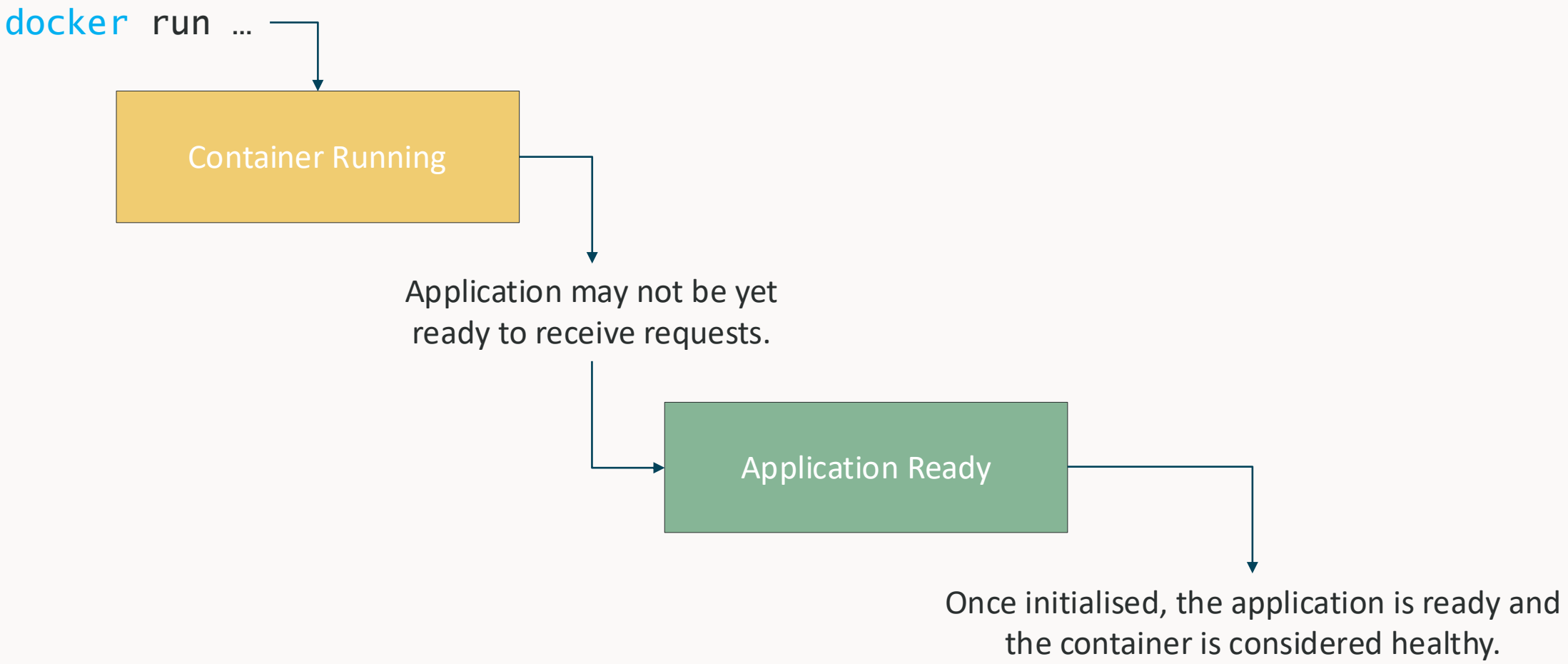
Handle the application lifecycle



Can the container start and stop safely?

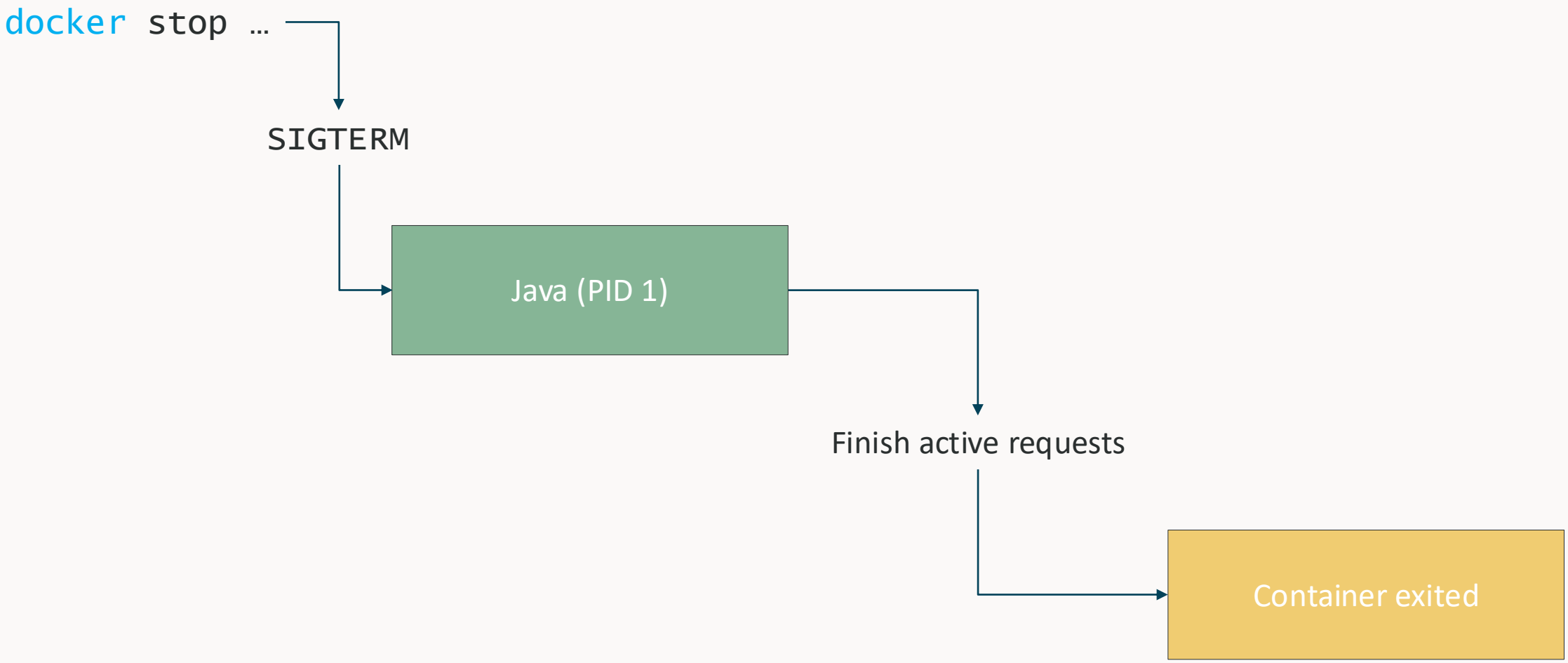
Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Handle the application lifecycle



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Handle the application lifecycle



Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Handle the application lifecycle

```yaml
spring:
  lifecycle:
    timeout-per-shutdown-phase: 15s


server:
  shutdown: graceful


management:
  endpoint:
    health:
      probes:
        enabled: true
```

Give the application enough time to shutdown.

Let’s the in-flight HTTP requests finish while the server stops accepting new ones.

Exposes liveness and readiness health endpoints

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Handle the application lifecycle

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

ARG JAR_FILE=./target/libs/application-*.jar
COPY ${JAR_FILE} app.jar

EXPOSE 8080

# The Oracle JDK image includes curl. Docker reports this container as
# healty when Spring Boot says it is ready to receive traffic.
HEALTHCHECK --interval=5s --timeout=3s --start-period=30s --retries=3 \
    CMD curl --fail --silent --output /dev/null \
        'http://127.0.0.1:8080/actuator/health/readiness' || exit 1

# Exec form keeps Java as PID 1 so docker stop delivers SIGTERM to the JVM.
ENTRYPOINT ["java", "-jar", "app.jar"]
```


Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Harden the running container



What access does the application need?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Harden the running container

```
docker run --entrypoint id 'container-registry.oracle.com/java/jdk:25.0.4.1'
uid=0(root) gid=0(root) groups=0(root)
```

The image does not specify a runtime user, so the container runs as root by default.

```
docker image inspect 'container-registry.oracle.com/java/jdk:25.0.4.1' \
     --format 'Baseline user={{.Config.User}}'
Baseline user=
```

The image has no default runtime user configured

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Harden the running container

```
FROM container-registry.oracle.com/java/jdk:25.0.4.1
WORKDIR /opt/app

# Keep application files owned by root and readable by the application user.
ARG JAR_FILE=./target/libs/application-*.jar
COPY --chown=0:0 --chmod=0444 ${JAR_FILE} app.jar

USER 10001:10001

ENTRYPOINT ["java", "-jar", "app.jar"]
```

This runs the Java application as a non-root user while keeping its JAR owned by root and readable by the application.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Harden the running container

```
docker run \
  --rm \
  --read-only \
  --tmpfs /tmp:rw,nosuid,nodev,size=64m,mode=1777 \
  --publish 8080:8080 \
  demo:application
```

Keep the root filesystem read-only and provide writable storage only where the application needs it.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Harden the running container

```
docker run \
  --rm \
  --read-only \
  --tmpfs /tmp:rw,nosuid,nodev,size=64m,mode=1777 \
  --mount "type=volume,source=[SOURCE],target=/diagnostics,volume-nocopy" \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-XX:StartFlightRecording=name=demo' \
  'demo:application'
```

Mount a writable volume at `/diagnostics` to preserve diagnostic files after the container is removed, while keeping its root filesystem read-only.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Harden the running container

```
docker run \
  --rm \
  --read-only \
  --cap-drop=ALL \
  --security-opt=no-new-privileges \
  --tmpfs /tmp:rw,nosuid,nodev,size=64m,mode=1777 \
  --mount "type=volume,source=[SOURCE],target=/diagnostics,volume-nocopy" \
  --publish 8080:8080 \
  --env 'JDK_JAVA_OPTIONS=-XX:StartFlightRecording=name=demo' \
  'demo:application'
```

Runs as non-root with restricted privileges, a read-only filesystem, memory limits, writable diagnostic storage.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Putting it all together



Is your container ready for production?

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Putting it all together

- Pin base images by digest, and update them deliberately to pick up patches.
- Separate dependencies from application code into container layers.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Putting it all together

- Include the Java modules and OS components you need, retaining diagnostic support.
- Generate AOT caches for the runtime and application you deploy, and measure the startup benefit.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Putting it all together

- Configure through environment variables, keeping secrets out of the image.
- Set CPU and memory limits explicitly, sizing the heap with room for native memory and other container usage.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Putting it all together

- Check readiness before sending traffic, and allow enough time for graceful shutdown.
- Run as a non-root user, with a read-only root filesystem and explicit writable paths where practical.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Putting it all together

- Keep diagnostics available, including somewhere to write and retrieve recordings and dumps.
- Measure optional optimisations, such as compact object headers, against your workload.

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com

Thank you

Albert Attard

Principal Java Success Consultant

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com



CFP open until Oct. 9th!

For more info: javaone.com

Copyright © 2026, Oracle and/or its affiliates | Confidential: Restricted | albert.attard@oracle.com


ORACLE